



## Capítulo 8 - Controle de passagens de nível

Avisar a aproximação, comandar a cancela e liberar a passagem somente depois de conferir a saída de toda a composição.

|                      |                                                                                                                                                                         |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Introdução           | Uma passagem de nível é o cruzamento de uma via rodoviária com a ferrovia no mesmo plano. Neste capítulo, vamos coordenar duas luzes de aviso e uma cancela de maquete. |
| Objetivo             | Organizar as etapas de aviso, fechamento, espera e abertura, utilizando temporização sem bloquear o programa.                                                           |
| Meta de aprendizagem | Ao final, você deverá conseguir simular uma aproximação, comandar a cancela, manter a sinalização durante a passagem e exigir confirmação para liberar o cruzamento.    |

### 1. O que será controlado?

Utilizaremos duas luzes vermelhas que piscam alternadamente e um servo que movimenta uma cancela. O exercício começa em simulação, com mensagens no Monitor Serial.

O modelo considera uma única composição, em um sentido conhecido, com confirmação manual de que todo o trem deixou o cruzamento. Ele não é um projeto para uma passagem de nível real.

O aviso e a cancela não controlam a tração. O programa deste capítulo não para uma locomotiva nem garante que ela aguarde o fechamento. A velocidade e a distância de aproximação precisam ser compatíveis com o tempo de aviso e de movimento da cancela.

### 2. Aproximação e liberação são informações diferentes

Um sensor antes do cruzamento pode indicar a aproximação da locomotiva. Essa informação inicia a sequência de aviso e fechamento.

Um sensor depois do cruzamento, acionado pelo ímã da locomotiva, não confirma que os últimos vagões já passaram. Por isso, não utilizaremos essa leitura isolada para abrir a cancela.

No exercício, o operador envia o comando L somente depois de conferir a saída de toda a composição e a ausência de outro trem aproximando-se.

Para uma liberação automática, será necessário definir uma detecção adequada da composição inteira ou outro método validado para o trecho. Um intervalo fixo após a passagem da locomotiva não resolve, sozinho, as diferenças de comprimento, velocidade ou parada do trem.



### 3. Etapas do funcionamento

| Etapa                | Ação do programa                                                 |
|----------------------|------------------------------------------------------------------|
| Passagem liberada    | Luzes apagadas e cancela comandada para aberta.                  |
| Aviso de aproximação | Luzes alternadas; aguarda o intervalo de aviso.                  |
| Fechamento           | Envia o comando de fechar e mantém as luzes.                     |
| Espera de liberação  | Mantém o comando de fechada e aguarda confirmação manual.        |
| Abertura             | Envia o comando de abrir, mantendo as luzes durante a espera.    |
| Bloqueio             | Comanda fechamento e mantém o aviso até conferência do operador. |

Os tempos de fechamento e abertura são estimativas para a bancada. Não substituem uma confirmação física da posição da cancela.

### 4. Estado inicial e tratamento de bloqueio

Ao iniciar, o programa fica bloqueado, com aviso ativo e comando de cancela fechada. O operador precisa conferir a montagem e a situação do cruzamento.

O comando R retira o bloqueio lógico, mas mantém a cancela comandada para fechada. Somente L, após a conferência de passagem livre, solicita a abertura.

O comando F ativa o bloqueio novamente. A recuperação não abre a cancela automaticamente.

Sem alimentação, o programa não consegue manter as luzes ou assegurar a posição mecânica do servo. A montagem física deve ser avaliada separadamente.



### 5. Montagem de bancada

| Elemento              | Ligação de exemplo                                        |
|-----------------------|-----------------------------------------------------------|
| Primeiro LED vermelho | GPIO 25 → resistor de 1 k $\Omega$ → anodo; catodo → GND. |
| Segundo LED vermelho  | GPIO 26 → resistor de 1 k $\Omega$ → anodo; catodo → GND. |
| Sinal do servo        | GPIO 18.                                                  |
| Alimentação do servo  | Fonte regulada de 5 V adequada ao servo.                  |
| Referência comum      | GND da fonte do servo ligado ao GND da ESP32.             |

Os GPIOs são exemplos para uma ESP32 clássica baseada no ESP32-WROOM. Não representam o mapeamento definitivo de uma gaveta da Ferrovia EFMN 74.

Alimente a ESP32 de bancada por USB. Não alimente o servo pelo pino 3V3 ou pela saída USB da placa. Nesta montagem, una os GNDs e não conecte o positivo da fonte do servo ao pino 5 V da ESP32.

Faça as ligações com a alimentação desligada. Inicie o teste do servo sem a haste conectada à cancela; ajuste o curso antes de instalar o mecanismo.

### 6. Temporização sem delay no acompanhamento

Utilizaremos `millis()` para contar o tempo de cada etapa e alternar os LEDs. Assim, o programa continua recebendo comandos durante o aviso e o movimento.

Os valores iniciais são 2 segundos de aviso, 1 segundo de espera de movimento e aproximadamente meio segundo para cada alternância das luzes. São valores didáticos, sujeitos a ajuste.

Os ângulos de abertura e fechamento também são exemplos. A geometria e a orientação do servo determinam os valores reais. Evite pressionar a haste contra um batente.



## 7. Comandos do exercício

| Comando | Utilização                                                        |
|---------|-------------------------------------------------------------------|
| A       | Simular aproximação do trem.                                      |
| L       | Confirmar manualmente cruzamento livre e solicitar abertura.      |
| F       | Ativar bloqueio e comandar fechamento.                            |
| R       | Após conferência, sair do bloqueio mantendo o comando de fechada. |

L só é aceito na etapa de espera com comando de fechada. Se for enviado durante o aviso ou o fechamento, será recusado e deverá ser reenviado depois da conferência.

Uma nova aproximação durante a abertura interrompe a liberação e envia novamente o comando de fechamento. Esse comportamento não substitui a detecção antecipada adequada.

## 8. Programa completo

Instale a biblioteca ESP32Servo pelo gerenciador de bibliotecas da Arduino IDE e crie `intermediate_08_level_crossing`. Mantenha `simulationOnly = true` no primeiro teste.

Copie todos os blocos de código apresentados abaixo, na ordem, para o mesmo programa.

```
#include <ESP32Servo.h>
const bool simulationOnly = true;
const int ledPins[2] = {25, 26};
const int servoPin = 18;
const int openAngle = 90;
const int closedAngle = 20;
const unsigned long warningTime = 2000;
const unsigned long movementTime = 1000;
enum CrossingStage { IDLE, WARNING, CLOSING, CLOSED, OPENING, FAULT };
CrossingStage stage = FAULT;
unsigned long stageStarted = 0;
Servo barrier;
```



```
void enterStage(CrossingStage nextStage) {
    stage = nextStage;
    stageStarted = millis();
    if (stage == CLOSING || stage == FAULT) {
        if (!simulationOnly) barrier.write(closedAngle);
    } else if (stage == OPENING) {
        if (!simulationOnly) barrier.write(openAngle);
    }
    switch (stage) {
        case IDLE: Serial.println("Crossing released"); break;
        case WARNING: Serial.println("Approach warning"); break;
        case CLOSING: Serial.println("Close command sent"); break;
        case CLOSED: Serial.println("Waiting for manual clearance"); break;
        case OPENING: Serial.println("Open command sent"); break;
        case FAULT: Serial.println("Blocked - check crossing"); break;
    }
}

void updateCrossing() {
    unsigned long now = millis();
    if (stage == WARNING && now - stageStarted >= warningTime) {
        enterStage(CLOSING);
    } else if (stage == CLOSING &&
        now - stageStarted >= movementTime) {
        enterStage(CLOSED);
    } else if (stage == OPENING &&
        now - stageStarted >= movementTime) {
        enterStage(IDLE);
    }
    if (!simulationOnly) {
        bool flash = (now / 500) % 2;
        bool warningActive = stage != IDLE;
        digitalWrite(ledPins[0], warningActive && flash);
        digitalWrite(ledPins[1], warningActive && !flash);
    }
}

void setup() {
    Serial.begin(115200);
    delay(1000);
    if (!simulationOnly) {
        pinMode(ledPins[0], OUTPUT);
        pinMode(ledPins[1], OUTPUT);
        barrier.setPeriodHertz(50);
        barrier.attach(servoPin);
    }
    Serial.println("EFMN 74 - Level crossing test");
    enterStage(FAULT);
    Serial.println("Approach: A | Clear: L | Block: F | Restore: R");
}
```



```
void loop() {  
  if (Serial.available() > 0) {  
    char command = Serial.read();  
    if (command == 'A') {  
      if (stage == IDLE) enterStage(WARNING);  
      else if (stage == OPENING) enterStage(CLOSING);  
    } else if (command == 'L') {  
      if (stage == CLOSED) enterStage(OPENING);  
      else Serial.println("Clearance not accepted in this stage");  
    } else if (command == 'F') {  
      enterStage(FAULT);  
    } else if (command == 'R') {  
      if (stage == FAULT) enterStage(CLOSED);  
    } else if (command != '\n' && command != '\r') {  
      Serial.println("Invalid command");  
    }  
  }  
  updateCrossing();  
}
```

## 9. Entendendo a sequência

enterStage() registra a nova etapa e seu instante inicial. Os comandos do servo são enviados na entrada das etapas de fechamento, abertura ou bloqueio.

updateCrossing() verifica o tempo transcorrido sem parar o programa. O else if permite avançar uma etapa por chamada. As luzes permanecem alternando enquanto a passagem não estiver liberada.

Depois de A, o programa passa pelo aviso e envia o fechamento. Após o tempo de movimento, aguarda L. Não existe abertura automática por tempo de permanência do trem.

O estado CLOSED significa que o comando de fechamento foi enviado e a espera terminou. Não existe sensor de fim de curso neste exercício. Antes de confirmar L, o operador também deve conferir a situação física da cancela.

## Exercício prático - Simulação

Meta: conferir a ordem das etapas e a necessidade de confirmação para abrir.

1. Utilize uma ESP32 de bancada por USB, com simulationOnly = true.
2. Compile, envie e abra o Monitor Serial em 115200 baud.
3. Pressione EN ou RESET e confira a mensagem de bloqueio.
4. Envie R e depois L. Aguarde a mensagem de passagem liberada.
5. Envie A. Confira o aviso, o comando de fechamento e a espera de liberação.
6. Aguarde alguns segundos sem enviar L. O programa deve continuar esperando.
7. Simule a conferência da saída de toda a composição e envie L.
8. Confira o comando de abertura e, depois, a passagem liberada.
9. Inicie outra aproximação com A e envie L durante o aviso. Confira a recusa.
10. Envie F durante qualquer etapa. Confira o bloqueio.



11. Envie R. O programa deverá voltar à espera de confirmação, sem abrir automaticamente.

Sequência esperada após A, partindo da passagem liberada:

Approach warning  
Close command sent  
Waiting for manual clearance

Após a confirmação L:

Open command sent  
Crossing released

Teste com LEDs e servo

1. Desligue a alimentação e confira a montagem do item 5.
2. Mantenha o servo sem a haste e altere `simulationOnly` para `false`.
3. Compile e envie. Observe o comando inicial de fechamento e as luzes alternadas.
4. Confira separadamente os ângulos de aberta e fechada e ajuste os valores no código.
5. Conecte a haste com a alimentação desligada e faça o ajuste fino sem esforço nos batentes.
6. Repita a sequência de aviso, fechamento, espera e abertura.
7. Confira que as luzes permanecem ativas durante a espera e a abertura e apagam somente na etapa liberada.

## 10. Integração futura com sensores

Um evento de aproximação reconhecido pelo programa de sensores poderá iniciar WARNING. A leitura deve ocorrer continuamente, com tempo suficiente para avisar e fechar antes da chegada ao cruzamento.

Para circulação nos dois sentidos, será necessário reconhecer a aproximação de cada lado. Para vários trens, será necessário manter o bloqueio enquanto qualquer aproximação ou ocupação continuar relevante.

O comando L não deve ser substituído diretamente por “sensor depois do cruzamento não acionado”. A liberação automática exige uma informação validada de que a composição inteira saiu e de que não há outro movimento conflitante.

Este programa não controla os relés de alimentação de uma gaveta, não verifica a posição mecânica da cancela e não está integrado à emergência da gaveta de controle principal.



### Resumo do capítulo

- A passagem de nível precisa de aviso antes do fechamento.
- A aproximação e a liberação são eventos diferentes.
- A passagem da locomotiva não confirma a saída dos últimos vagões.
- `millis()` permite acompanhar tempos sem bloquear o programa.
- As luzes permanecem ativas durante fechamento, espera e abertura.
- O bloqueio exige conferência e não abre automaticamente ao ser retirado.
- Os tempos e os ângulos precisam de ajuste na montagem.
- Comando de posição não é confirmação mecânica.

Próximo capítulo: Controle de velocidade.

### Referência técnica

Biblioteca ESP32Servo: documentação e exemplos oficiais em  
<https://github.com/madhephaestus/ESP32Servo>